Practical 1

Title: Managing Data Frames with the dplyr package – Use Select , Filter , Rename

Problem: Use dplyr Grammar for inbuilt data set car **- Use Select , Filter , Rename**

Question 1: Select specific columns and filter rows

Question: Select the columns `mpg`, `cyl`, and `hp`, and filter the rows where `mpg` is greater than 25.

R Code:

```
library(dplyr)
selected_filtered_data <- mtcars %>% select(mpg, cyl, hp) %>% filter(mpg >
25)
selected_filtered_data
```

Expected Outcome: A data frame showing only the cars with `mpg` greater than 25 and the selected columns.

```
                   mpg cyl  hp
Fiat 128          32.4   4  66
Honda Civic       30.4   4  52
Toyota Corolla    33.9   4  65
Fiat X1-9         27.3   4  66
Lotus Europa      30.4   4 113
```

Question 2: Rename columns and filter rows

Question: Rename the column `wt` to `Weight` and filter the rows where `Weight` is less than 2.5.

R Code:

```
renamed_filtered_data <- mtcars %>% rename(Weight = wt) %>% filter(Weight <
2.5)
renamed_filtered_data
```

Expected Outcome: A data frame showing the renamed column and only the cars with `Weight` less than 2.5.

```
                   mpg cyl  disp  hp drat Weight  qsec vs am gear carb
Honda Civic       30.4   4  75.7  52 4.93  1.615 18.52  1  1    4    2
```

```
Fiat X1-9         27.3   4  79.0  66 4.08  1.935 18.90  1  1    4    1
Lotus Europa      30.4   4  95.1 113 3.77  1.513 16.90  1  1    5    2
```

Question 3: Filter rows and select specific columns

Question: Filter the rows where `cyl` is 6 and select the columns `mpg`, `hp`, and `drat`.

R Code:

```
filtered_selected_data <- mtcars %>% filter(cyl == 6) %>% select(mpg, hp,
drat)
filtered_selected_data
```

Expected Outcome: A data frame showing only the cars with `cyl` equal to 6 and the selected columns.

```
                   mpg  hp drat
Mazda RX4         21.0 110 3.90
Mazda RX4 Wag     21.0 110 3.90
Hornet 4 Drive    21.4 110 3.08
Valiant           18.1 105 2.76
Merc 280          19.2 123 3.92
Merc 280C         17.8 123 3.92
```

Question 4: Rename columns and filter rows based on multiple conditions

Question: Rename the column `hp` to `HorsePower`, filter the rows where `HorsePower` is greater than 150 and `mpg` is greater than 15.

R Code:

```
renamed_filtered_data <- mtcars %>% rename(HorsePower = hp) %>%
filter(HorsePower > 150 & mpg > 15)
renamed_filtered_data
```

Expected Outcome: A data frame showing the renamed column and only the cars with `HorsePower` greater than 150 and `mpg` greater than 15.

```
                   mpg cyl  disp HorsePower drat    wt  qsec vs am gear carb
Hornet Sportabout 18.7   8 360.0        175 3.15 3.440 17.02  0  0    3    2
Duster 360        14.3   8 360.0        245 3.21 3.570 15.84  0  0    3    4
Camaro Z28       13.3   8 350.0        245 3.73 3.840 15.41  0  0    3    4
```

```
Ford Pantera L   15.8   8 351.0        264 4.22 3.170 14.50  0  1    5    4
Maserati Bora    15.0   8 301.0        335 3.54 3.570 14.60  0  1    5    8
```

Question 5: Filter rows, select specific columns, and rename

Question: Filter the rows where `gear` is 4, select the columns `mpg`, `cyl`, and `gear`, and rename `gear` to `Gears`.

R Code:

```
filtered_selected_renamed_data <- mtcars %>% filter(gear == 4) %>%
select(mpg, cyl, gear) %>% rename(Gears = gear)
filtered_selected_renamed_data
```

Expected Outcome: A data frame showing only the cars with `gear` equal to 4, the selected columns, and the renamed column.

```
                   mpg cyl Gears
Mazda RX4         21.0   6     4
Mazda RX4 Wag     21.0   6     4
Datsun 710        22.8   4     4
Merc 240D         24.4   4     4
Merc 230          22.8   4     4
Fiat 128          32.4   4     4
Honda Civic       30.4   4     4
Toyota Corolla    33.9   4     4
Fiat X1-9         27.3   4     4
Porsche 914-2     26.0   4     5
Lotus Europa      30.4   4     5
Ford Pantera L    15.8   8     5
Ferrari Dino      19.7   6     5
Maserati Bora     15.0   8     5
```

Practical 2

Title: Managing Data Frames with the dplyr package – Use mutate , group by, summarise

Problem: Use dplyr Grammar for inbuilt data set

Example 1: Create a new column using `mutate`

Problem: Create a new column `power_to_weight` as the ratio of `hp` to `wt`.

R Code:

```
library(dplyr)
mtcars <- mtcars %>% mutate(power_to_weight = hp / wt)
head(mtcars)
```

Expected Outcome: A data frame with the first few rows showing the new `power_to_weight` column.

```
                   mpg cyl  disp  hp drat    wt  qsec vs am gear carb
power_to_weight
Mazda RX4         21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
41.98473
Mazda RX4 Wag     21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
38.26087
Datsun 710        22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
40.08621
Hornet 4 Drive    21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
34.21550
Hornet Sportabout 18.7   8 360.0 175 3.15 3.440 17.02  0  0    3    2
50.87209
Valiant           18.1   6 225.0 105 2.76 3.460 20.22  1  0    3    1
30.34682
```

Example 2: Group by a column and summarise

Problem: Calculate the average `mpg` for each number of cylinders (`cyl`).

R Code:

```
average_mpg <- mtcars %>% group_by(cyl) %>% summarise(avg_mpg = mean(mpg))
average_mpg
```

Expected Outcome: A data frame showing the average `mpg` for each number of cylinders.

```
# A tibble: 3 × 2
    cyl avg_mpg
  <dbl>   <dbl>
1     4    26.7
2     6    19.7
3     8    15.1
```

Example 3: Group by and summarise multiple columns

Problem: Calculate the average `mpg` and `hp` for each number of cylinders (`cyl`).

R Code:

```
average_mpg_hp <- mtcars %>% group_by(cyl) %>% summarise(avg_mpg = mean(mpg),
avg_hp = mean(hp))
average_mpg_hp
```

Expected Outcome: A data frame showing the average `mpg` and `hp` for each number of cylinders.

```
# A tibble: 3 × 3
    cyl avg_mpg avg_hp
  <dbl>   <dbl>  <dbl>
1     4    26.7   82.6
2     6    19.7  122.
3     8    15.1  209.
```

Example 4: Create a new column, group by, and summarise

Problem: Create a new column `hp_per_cyl` as the ratio of `hp` to `cyl`, then calculate the average `hp_per_cyl` for each number of gears (`gear`).

R Code:

```
hp_per_cyl_summary <- mtcars %>% mutate(hp_per_cyl = hp / cyl) %>%
group_by(gear) %>% summarise(avg_hp_per_cyl = mean(hp_per_cyl))
hp_per_cyl_summary
```

Expected Outcome: A data frame showing the average `hp_per_cyl` for each number of gears.

```
# A tibble: 3 × 2
   gear avg_hp_per_cyl
  <dbl>          <dbl>
1     3           37.4
2     4           28.2
3     5           48.4
```

Example 5: Filter, mutate, group by, and summarise

Problem: Filter the rows where `am` is 1 (manual transmission), create a new column `mpg_per_hp` as the ratio of `mpg` to `hp`, and calculate the average `mpg_per_hp` for each number of cylinders (`cyl`).

R Code:

```
manual_mpg_per_hp <- mtcars %>% filter(am == 1) %>% mutate(mpg_per_hp = mpg /
hp) %>% group_by(cyl) %>% summarise(avg_mpg_per_hp = mean(mpg_per_hp))
manual_mpg_per_hp
```

Expected Outcome: A data frame showing the average `mpg_per_hp` for each number of cylinders for cars with manual transmission.

```
# A tibble: 3 × 2
    cyl avg_mpg_per_hp
  <dbl>          <dbl>
1     4         0.353
2     6         0.158
3     8         0.0738
```

Practical 3

Title: Data Preparation

Problem: Use the data set air quality from inbuilt data sets library.

a) Use summary statistics and find the important key values from the output

b) Use boxplot and find the interquartile range. Interpret the boxplot and inner and outer fencing of outliers

c) Check the missing value in the data set and fine the suitable solution for the missing values.

Part a: Summary Statistics

Problem: Use summary statistics and find the important key values from the output.

R Code:

```
# Load the airquality dataset
data("airquality")

# Summary statistics
summary_stats <- summary(airquality)
summary_stats
```

Expected Outcome: The summary statistics will provide key values such as the minimum, 1st quartile, median, mean, 3rd quartile, and maximum for each column in the dataset.

```
     Ozone           Solar.R           Wind             Temp           Month
Day
 Min.   :  1.00   Min.   :  7.0   Min.   : 1.700   Min.   :56.00   Min.
:5.000   Min.   : 1.0
 1st Qu.: 18.00   1st Qu.:115.8   1st Qu.: 7.400   1st Qu.:72.00   1st
Qu.:6.000   1st Qu.: 8.0
 Median : 31.50   Median :205.0   Median : 9.700   Median :79.00   Median
:7.000   Median :16.0
 Mean   : 42.13   Mean   :185.9   Mean   : 9.958   Mean   :77.88   Mean
:6.993   Mean   :15.8
 3rd Qu.: 63.25   3rd Qu.:258.8   3rd Qu.:11.500   3rd Qu.:85.00   3rd
Qu.:8.000   3rd Qu.:23.0
 Max.   :168.00   Max.   :334.0   Max.   :20.700   Max.   :97.00   Max.
:9.000   Max.   :31.0
 NA's   :37       NA's   :7       NA's   :0        NA's   :0       NA's   :0
NA's   :0
```

Part b: Boxplot and Interquartile Range

Problem: Use boxplot and find the interquartile range. Interpret the boxplot and inner and outer fencing of outliers.

R Code:

```
# Boxplot for each variable
boxplot(airquality, main = "Boxplot of Air Quality Dataset", col =
"lightblue")

# Calculate the interquartile range (IQR) for each column
iqr_values <- apply(airquality, 2, function(x) IQR(x, na.rm = TRUE))
iqr_values
```

Expected Outcome: The boxplot will display the distribution of each variable, showing the median, quartiles, and potential outliers. The interquartile range (IQR) values will be calculated for each column.

```
    Ozone   Solar.R      Wind      Temp     Month       Day
  45.5000   142.000     3.400    13.000     2.000    15.000
```

Interpretation:

- **Boxplot:** The boxplot shows the spread of the data for each variable. The box represents the interquartile range (IQR), which contains the middle 50% of the data. The line inside the box

represents the median. The whiskers extend to the smallest and largest values within 1.5 * IQR from the quartiles. Points outside this range are considered outliers.

- **Inner Fencing:** Values within 1.5 * IQR from the quartiles.
- **Outer Fencing:** Values beyond 3 * IQR from the quartiles.

Part c: Check for Missing Values and Handle Them

Problem: Check the missing values in the dataset and find a suitable solution for the missing values.

R Code:

```
# Check for missing values
missing_values <- colSums(is.na(airquality))
missing_values

# Fill missing values with the mean of each column
airquality_filled <- airquality
for(i in 1:ncol(airquality_filled)) {
  airquality_filled[is.na(airquality_filled[, i]), i] <-
mean(airquality_filled[, i], na.rm = TRUE)
}

# Check for missing values after filling
filled_missing_values <- colSums(is.na(airquality_filled))
filled_missing_values
```

Expected Outcome: The number of missing values for each column before and after filling them with the mean.

```
# Missing Values in the Dataset:
 Ozone    Solar.R       Wind       Temp      Month        Day
    37          7          0          0          0          0

# Missing Values after filling with mean:
 Ozone    Solar.R       Wind       Temp      Month        Day
     0          0          0          0          0          0
```